Robust and Compact Recognition of Code-Switched Speech

Rui Cong and Phoebe and Nibras and Bridgette
School of Computing and Information Systems, Singapore Management University

Abstract

Speech recognisers are scored by one overall error rate. Yet in English–Mandarin code-switched speech, the few words from the other language (*embedded* words) often carry the meaning, and they may be the first to suffer when a model is shrunk to run on a device. We split Whisper's errors by each word's role in its sentence, on two corpora (ASCEND and NSC Part 4) and 120 recordings of our own, and find three things. First, the same word is wrong more often when it is embedded; fine-tuning removes this penalty for Mandarin but not for English. Second, two training losses aimed at embedded words, and language-ID re-ranking applied only to clips with a switch, give small but statistically reliable gains (best system: 20.6% error on NSC, down from 55.1%). Third, 8-bit quantization causes no detectable harm, while 4-bit hurts embedded words more than other words in the smaller models; our repairs recover about half to four-fifths of that loss in NF4, but not in the format used on device. We therefore recommend 8-bit models for on-device use.

1 Introduction

Many Singaporeans switch between English and Mandarin within one sentence. In "我明天要交 assignment 啦" ("I have to submit my assignment tomorrow"), Mandarin is the *matrix* language and "assignment" is an *embedded* word. Embedded words are often the ones that matter most: a drug name, a deadline, a course code or a place.

Clinics, counselling and live captions for deaf users often cannot send audio to the cloud, so the recogniser must run on the user's own device. To make a model small enough, its weights are usually *quantized*: stored in 8 or 4 bits instead of 16. Quantized models are judged by one overall error rate. Embedded words are a small share of all words, so they can fail badly while that rate barely moves.

Fine-tuning Whisper (Radford et al., 2023) on code-switched Mandarin–English speech improves accuracy (Yang et al., 2024; Zhou et al., 2025). Ugan et al. (2025b) give switch points extra weight in training, and PIER (Ugan et al., 2025a) scores errors on code-switched words separately. Quantization of Whisper has been studied on monolingual speech (Söhler et al., 2026; Ferraz, 2024). No work we know of asks how quantization changes errors on embedded words, or whether those words fail because of their language or because of their role in the sentence.

We study Whisper tiny, base and small on two corpora with opposite majority languages: ASCEND (Lovenia et al., 2022), mostly Mandarin-main, and the Mandarin-speaker part of NSC Part 4 (Koh et al., 2019), mostly English-main. We ask three questions, each with the hypothesis we started from:

- **RQ1**: Do embedded words fail because of their role or their language? (H1: partly because of their role.)
- **RQ2**: Do training and decoding aimed at embedded words reduce their errors at 16-bit? (H2: yes, without raising overall error.)
- **RQ3**: Does quantization hurt embedded words more than other words, and can this be repaired? (H3: yes, most in the smallest models.)

Our answers, in one line each:

- **RQ1, role matters.** The same word is wrong more often when it is embedded. Fine-tuning removes this penalty for embedded Mandarin but not for embedded English (§7).

- **RQ2, small but reliable gains.** A role-weighted loss plus a semantic minimum-risk loss lowers both overall and embedded-word error on NSC, and language-ID re-ranking helps when used only on clips with a switch (§8).
- **RQ3, use 8-bit.** 8-bit causes no detectable harm. 4-bit hurts embedded words more than other words in tiny and base. Our two repairs work in NF4 but not in the format used on device (§9).

Our best system, Whisper small fine-tuned with both losses, lowers the overall error on NSC from 55.1% to 20.6% and the embedded-word error from 76.7% to 21.3%. Stored in 8-bit with whisper.cpp (Gerganov et al.), it is a 252 MB file with a real-time factor of 0.57 on a laptop CPU. Code, scoring tools and all result tables are at `https://github.com/nibess1/cs425_codeswitch`.

2 Related Work

Whisper on code-switched speech. Whisper (Radford et al., 2023) is trained on weakly labelled multilingual speech. Fine-tuning it on code-switched corpora lowers its error rate considerably (Yang et al., 2024; Zhou et al., 2025). Even at full precision, it can drop embedded words or output their translation (Peng et al., 2023). These studies report one overall error rate.

Scoring and training for embedded words. PIER (Ugan et al., 2025a) computes the error rate only on words of interest, such as the code-switched words, and shows that an overall rate can improve while those words get worse. Ugan et al. (2025b) give tokens at switch points extra weight in training. Minimum word error rate training (Prabhavalkar et al., 2018) lowers the expected error over a set of candidate transcripts instead of the error of each token.

Quantizing speech models. Weights can be stored in 8 bits (Dettmers et al., 2022) or in 4 bits with NF4, and low-rank adapters (Hu et al., 2022) can be trained on top of a 4-bit model (Dettmers et al., 2023). For Whisper, Söhler et al. (2026) study how quantization choices affect error, and Ferraz (2024) find that compression hurts low-resource languages more. On-device runtimes such as CTranslate2 (OpenNMT) and whisper.cpp (Gerganov et al.) have their own formats. None of these studies looks at code-switched speech.

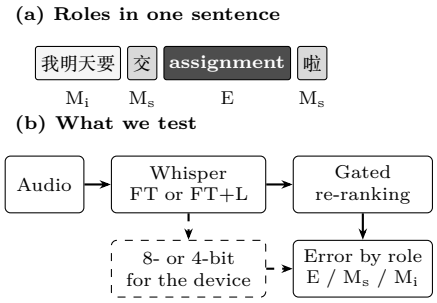


Figure 1: (a) Every unit gets a role: embedded (E), matrix next to a switch (M_s) or other matrix (M_i). (b) Fine-tuned Whisper (RQ1–2), gated re-ranking (RQ2) and quantized models (RQ3) are all scored by role.

What we change. We start from Whisper fully fine-tuned on code-switched speech, as in Yang et al. (2024) and Zhou et al. (2025), and add:

- scoring by three roles (embedded, matrix next to a switch, other matrix), where PIER scores the words of interest only;
- separate loss weights for embedded words and for matrix words next to a switch, where Ugan et al. (2025b) weight switch points;
- a minimum-risk loss that forgives correct translations, where MWER counts every word alike;
- re-ranking with a frame-level language-ID head, used only when a switch is heard;
- a measure of how much more quantization hurts embedded words, in on-device formats, and two repairs.

3 Task and Metrics

Units and roles. A shared text normaliser turns every reference and output into *units*: one Chinese character, one English word or one digit string. The matrix language of a sentence is the one with more words, after segmenting the Chinese part into words; sentences with a tie count towards the overall error rate only. Each unit gets one role (Figure 1a): *embedded* (E), a unit in the other language; *matrix at a switch* (M_s), a matrix unit directly before or after a language change; or

matrix elsewhere (M_i). In the text we call embedded units *embedded words*.

Error rates. The mixed error rate is

$$\mathrm{MER} = \frac{S + D + I}{N}, \tag{1}$$

where S, D and I are the numbers of substituted, deleted and inserted units in the best alignment of output and reference, and N is the number of reference units. A second alignment pass charges each block of errors, including its insertions, to the reference units it covers. The error rate of a role is the errors charged to it divided by its number of units, so it can exceed 100%. We also report the *critical-word error rate* (CWER), the share of names, numbers, dates, places and course codes not reproduced exactly.

Fragility and recovery. To ask whether quantization hurts embedded words more than matrix words, we compute on code-switched utterances

$$R = \frac{\Delta_{\mathrm{E}}}{\Delta_M}, \tag{2}$$

where Δ_{E} is the change in embedded error (points) from the 16-bit model to the quantized one, and Δ_M the same change for matrix units (M_s and M_i together). $R > 1$ means quantization hurts embedded words more. Each quantized model is compared with a 16-bit run of the *same* backend. R is reported only when the interval of Δ_M excludes 0, since otherwise the ratio is unstable. For a repair applied to a quantized model,

$$\mathrm{Rec} = \frac{E_q - E_{q,\mathrm{fix}}}{E_q - E_{16}}, \tag{3}$$

where E_{16}, E_q and $E_{q,\mathrm{fix}}$ are the embedded errors of the 16-bit model, the quantized model, and the quantized model with the repair. Rec = 1 means the whole quantization loss is removed and 0 that nothing is. Rec is reported only where the loss $E_q - E_{16}$ has an interval excluding 0.

Confidence intervals. All intervals are 95% bootstrap intervals over 1,000 resamples, paired when two systems are compared. For NSC we resample whole speakers, which gives wider intervals; these are the ones we quote. ASCEND test has only two speakers, so its utterance-level intervals are optimistic.

Set	Utts	Hours	CS%	E units	Spk
ASCEND train	9,869	8.8	28	7,081	18
ASCEND val	1,130	0.9	18	471	3
ASCEND test	1,315	0.9	28	772	2
NSC train	35,647	40.0	62	105,493	412
NSC val	3,367	3.0	25	4,099	60
NSC test (Same)	8,876	8.0	29	11,042	120
NSC test (Diff)	4,085	4.0	29	5,648	626
Ours (phone)	120	0.18	100	227	3

Table 1: Data. CS% = share of utterances that contain a switch; Spk = speakers. NSC = NSC Part 4, Mandarin speakers; Same = Same Room, Diff = Different Room (Zoom). Ours = our phone recordings (test only).

4 Data

Corpora. ASCEND (Lovenia et al., 2022) is spontaneous Mandarin–English conversation; 69% of its sentences are Mandarin-main. NSC Part 4 (Koh et al., 2019) is conversational Singapore speech in which speakers were asked to switch. We keep its Mandarin-speaking participants, found from the speaker information sheet; 70% of their sentences are English-main, but about 4 in 10 of their code-switched sentences are Mandarin-main, so NSC also contains many embedded English words. NSC has two recording conditions: *Same Room*, both speakers in one room, and *Different Room*, over Zoom. Table 1 gives the splits.

Sampling and splits. NSC training is capped at 40 h of the 193 h of usable Same Room speech: code-switched utterances first (up to 30 h), then English-only ones. Validation and test are random samples within a session-level speaker split. The Different Room test is a random 4 h sample, and none of its speakers appears in training. ASCEND validation and test speakers do not appear in training either. NSC code-switched utterances have more switches than ASCEND ones (2.85 vs. 2.12 per utterance, Different Room) and longer embedded spans (2.8 vs. 2.0 units).

Our phone recordings. To test critical words in real voices, we wrote 120 code-switched sentences dense in names, numbers, drug and course terms (*my* 密码 *has twelve characters now*), 60 English-main and 60 Mandarin-main. Three authors, English-speaking Singaporeans of Chinese background, each read a different 40 (20 of each kind), once,

on their own phone in a quiet room. The 48 kHz files go through the same pipeline as the corpora (16 kHz mono); clips last 3.8–7.2 s, 10.7 min in total. Every clip follows its sentence exactly, so the sentence is the reference.

Labels. Word times come from forced alignment with MMS (Pratap et al., 2024) on romanised text (Hermjakob et al., 2018); on FLEURS (Conneau et al., 2023) clips spliced at known points, the aligned switch is a median of 10 ms from the true one. An embedded word replaced by text in the other script counts as a *translation* if the pair is in CC-CEDICT (MDBG) or its LaBSE similarity (Feng et al., 2022) is at least $\tau = 0.65$. Otherwise it counts as a *transliteration* (a sound-alike) if its phonetic similarity (pinyin and Metaphone codes compared by edit distance) is at least $\theta = 0.1$. Both thresholds were chosen on 1,768 ASCEND pairs labelled by two LLM annotators and adjudicated ($\kappa = 0.84$ for translation vs. the rest). The critical-word list combines a rule tagger with a list labelled the same way ($\kappa = 0.77$). Two small human audits of these labels are reported in Limitations.

5 Methods

We compare four kinds of system: *off-the-shelf* Whisper; *FT*, Whisper fine-tuned with the ordinary loss; *FT+L*, fine-tuned with our two losses (a, b); and FT or FT+L with *gated re-ranking* (c, d). The repairs (e) are applied to quantized FT and FT+L models.

(a) Role-weighted loss (RQ2). Ordinary fine-tuning gives every output token the same cross-entropy loss. We give tokens of embedded words, and of matrix words next to a switch, extra weight:

$$\mathcal{L}_{\text{role}} = -\sum_{t} \big(1 + \alpha\, e_t + \beta\, s_t\big) \log p_\phi(y_t \mid y_{<t}, x), \tag{4}$$

where y_t is the t-th reference token, x the audio, p_ϕ the model's probability, e_t and s_t are 1 if token t belongs to an embedded unit or to a matrix unit at a switch (else 0), and α, β are the extra weights ($\alpha = \beta = 0$ is ordinary fine-tuning). A token's role is found from the bytes it covers, which is exact even when one Chinese character is split across two tokens.

(b) Semantic minimum-risk loss (RQ2). Following minimum word error rate training (Prabhavalkar et al., 2018), we lower the expected cost over a set $\mathcal{C}$ of candidate transcripts for each clip:

$$\mathcal{L}_{\text{sem}} = \sum_{y \in \mathcal{C}} \hat{p}_\phi(y \mid x)\, c(y), \tag{5}$$

where $\hat{p}_\phi(y \mid x)$ is the model's probability of candidate y, renormalised over $\mathcal{C}$, and $c(y)$ counts its unit errors, except that a correct translation of a non-critical embedded word costs $\rho_0 < 1$ instead of 1. $\mathcal{C}$ pools beam-4 outputs under a forced Mandarin, a forced English and a free language token, plus one candidate per embedded word with that word swapped for its dictionary translation. Candidates are regenerated with the current model every four steps. The training loss of FT+L is $\mathcal{L} = \mathcal{L}_{\text{role}} + \lambda\, \mathcal{L}_{\text{sem}}$, where λ weights the semantic term. We designed this loss expecting many translation errors; RQ1 shows they are rare, so in practice it acts as a sequence-level minimum-risk loss.

(c) Language-ID head (RQ2). A two-layer classifier on Whisper's frozen encoder labels every 20 ms frame as English, Mandarin or non-speech. It is trained on ASCEND and NSC frames labelled by forced alignment, plus 6,000 FLEURS clips spliced from word-sized pieces (15.9 h), some joining two speakers so that a change of speaker is not taken as a change of language. Output is smoothed with a 5-frame median filter, and language runs under 200 ms are merged into their neighbours.

(d) Gated re-ranking (RQ2). For each clip we pool up to 12 candidates (beam 4 under each of the three language tokens), drop those that repeat themselves (gzip compression ratio above 2.4, Whisper's own check), and pick one by a weighted sum of three scores: Whisper's log-probability, an acoustic match from the MMS model, and agreement between each word's language and the language-ID head over that word's frames. Weights are tuned on validation. The *on-device* variant drops MMS, which is larger than Whisper small. *Gating*: a clip is re-ranked only when the head hears at least θ_{gate} ms of the minority language; otherwise the greedy output is kept.

(e) Quantization repairs (RQ3). *Mixed precision*: we quantize one Transformer block at a time to NF4, rank blocks by how much the output on embedded-word tokens changes (KL divergence), and keep the top k blocks at 16-bit. *Quantization-aware fine-tuning*: QLoRA (Dettmers et al., 2023) trains 16-bit low-rank adapters on top of the NF4 model with the model's own loss. We test the adapters as they are, and merged into the weights and re-quantized for whisper.cpp. A further attempt, a direct penalty on transliterations, is described in Appendix A.6.

6 Experimental Setup

Model. Whisper (Radford et al., 2023) is a Transformer encoder–decoder. The encoder reads an 80-bin log-Mel spectrogram of a 30 s window (all our clips are shorter, so no chunking is needed); the decoder writes byte-level BPE tokens, starting with a language token. We use tiny (39M parameters, 4+4 layers), base (74M, 6+6) and small (244M, 12+12), on 16 kHz mono audio.

Training. We fine-tune tiny, base and small on ASCEND and NSC training data together, updating all weights (Table 2). The loss weights were chosen on base by grid search on validation ($\alpha \in \{0.5, 1, 2\}$, $\beta \in \{0, 0.5\}$, then $\lambda \in \{0.1, 0.3\}$, $\rho_0 \in \{0.2, 0.5\}$) and used unchanged for small. The gate threshold was chosen from $\theta_{\text{gate}} \in \{0, 200, 400, 800, \infty\}$ ms; 200 ms won for every model. Every selection rule was written down before any test result was seen. Each system is trained once; retraining the same base system with the same settings moved test MER by at most 0.2 points (Appendix A.6).

Quantization. On GPU we use bitsandbytes LLM.int8 (Dettmers et al., 2022) and NF4 on all linear layers, compared with the 16-bit model. On CPU we use CTranslate2 int8 (compared with CTranslate2 float32) and whisper.cpp Q8_0, Q5_0 and Q4_0 (compared with whisper.cpp F16). CPU rows use a fixed subset of the test sets: all of ASCEND test, 4,000 NSC Same Room and 2,000 Different Room utterances.

Decoding, hardware and code. Decoding is greedy with a 128-token cap unless stated.

Setting	Value
Optimiser	AdamW, $\beta = (0.9, 0.98)$, $\epsilon = 10^{-6}$, weight decay 0.01, gradient clip 1.0
Learning rate	tiny 3.75×10^{-5}, base 2.5×10^{-5}, small 1.25×10^{-5}; 500 warm-up steps, then linear decay to 0
Batch, epochs	32 utterances (small: 16×2 accumulation); 3 epochs; bf16; seed 425
Targets	main-language token first; at most 128 tokens
Checkpoint	lowest mean val MER, checked every 1,000 steps on 600 utterances per val set
Role loss	$\alpha = 0.5$, $\beta = 0.5$
Sem. loss	$\lambda = 0.1$, $\rho_0 = 0.5$; candidates every 4 steps on a sub-batch of 8 (small: 4)
LID head	2 layers on the frozen encoder; AdamW, one-cycle, peak lr 10^{-3}; 2 epochs; batch 32
Re-ranking	≤ 12 candidates (beam 4×3 language prefixes); gzip ratio ≤ 2.4; gate $\theta_{\text{gate}} = 200$ ms
Mixed prec.	blocks kept at 16-bit: tiny FT 2, base FT+L 3 (chosen on val)
QLoRA	rank 16, scale 32, dropout 0.05 on all attention and MLP projections; lr 10^{-4}, 100 warm-up steps, 1 epoch
Thresholds	translation $\tau = 0.65$; transliteration $\theta = 0.1$
Decoding	greedy, ≤ 128 tokens; language token restricted to zh/en (HF, CTranslate2)

Table 2: Key hyper-parameters. Learning rates and batch sizes are per model; loss weights were tuned on base (Appendix A.1) and reused unchanged.

Hugging Face and CTranslate2 restrict the language token to Mandarin or English; whisper.cpp chooses among all languages. Training used one NVIDIA L40S GPU. Speed is measured on a MacBook Pro (Apple M2 Pro, 4 threads) on 100 test clips (259 s). All code is in one repository; each step (pre-processing, training, testing, scoring) reads and writes plain files and can be rerun on its own, and `score.py` produces every number in this report. Appendix A.9 lists the commands.

7 RQ1: Role or Language?

Answer: role matters, and fine-tuning fixes it only for embedded Mandarin (H1 supported).

Embedded words fail most. In almost every system and test set, embedded words have the highest error, then matrix words at a switch, then other matrix words (Figure 2).

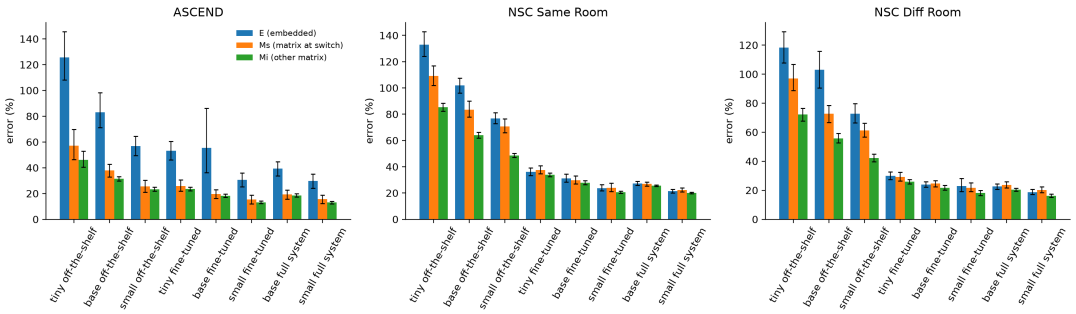


Figure 2: Error rate by role (E, M_s, M_i) for each system on the three test sets. Embedded words have the highest error in almost every system.

System	English word	Chinese character
small, off-the-shelf	**+15.4**	**+26.0**
base, off-the-shelf	**+19.7**	**+32.0**
base, FT	**+8.0**	**+1.3**
small, FT	**+7.4**	+0.3

Table 3: The same word, embedded vs. in the matrix language: error when embedded minus error in the matrix language (points), over word types seen in both roles. Bold: 95% CI excludes 0.

Off-the-shelf small gets 56.8% of embedded words wrong on ASCEND against 23.4% of M_i words, and 76.7% on NSC against 48.6%.

The same word is harder when embedded. Table 3 compares each word type that occurs in both roles. For off-the-shelf small, an English word is 15.4 points [12.1, 18.6] more likely to be wrong when embedded than in an English-main sentence, and a Chinese character 26.0 points [23.9, 27.8] more. After fine-tuning, the gap for Chinese characters disappears (+0.3 for small) but the gap for English words remains (+7.4 [5.2, 10.1]). The pattern is the same within NSC alone (small, FT: English +7.2 [5.0, 9.9], Chinese −2.0 [−3.1, −0.9]), so it is not a difference between corpora; ASCEND alone is too small to tell (results table 11b).

A logistic regression on 46,916 units from 122 speakers confirms this after controlling for language, training-data frequency, spoken duration and corpus (Appendix A.8). Off the shelf, being embedded adds about one unit of log-odds of error (small: 0.94 [0.78, 1.15]). After fine-tuning this term is about zero, but the extra term for embedded English stays large (small: 0.79 [0.42, 1.11]).

Embedded words are rarely translated. We expected many embedded words to be replaced by their translation. Fine-tuned models do this to only 0.5–1.9% of embedded words (off-the-shelf: 3.4–4.0%), for every translation threshold we tried (Appendix A.3). Some errors instead write an English word in Chinese characters that sound like it (deadline→得来, python→派森); our detector flags up to 10.9% of embedded words this way. Caveat: this count is likely too high. In a human audit of 26 pairs, none was a translation, but 11 of the 13 pairs our labels call sound-alikes were judged unrelated (§12).

The same holds on our phone recordings. On the 120 recordings, off-the-shelf small gets 83.9% of embedded words wrong but only 23.5% of M_i words, and fine-tuning (small, FT) cuts embedded error by 52.5 points [39.8, 65.3]. Matrix words next to a switch are harder than other matrix words in four of five fine-tuned models (small, FT+L: 12.4% vs. 9.0%).

8 RQ2: Methods at 16-bit

Answer: yes, but the gains are small; they are reliable on NSC and within noise on ASCEND (H2 partly supported).

Fine-tuning matters most. Fine-tuning on ASCEND and NSC brings by far the largest gain, most of all on NSC, where off-the-shelf small has 55.1% MER and small FT 21.4% (Table 4).

Our losses help on NSC. Both losses lower embedded error, and together they work best (Table 5a). FT+L is significantly bet-

	ASCEND		NSC Same Room		NSC Different Room	
System	MER	E	MER	E	MER	E
small, off-the-shelf	26.3	56.8	55.1	76.7	48.8	72.7
tiny, FT	26.1	53.2	34.5	36.2	26.8	29.8
base, FT	21.3	55.3	28.4	31.0	22.4	24.0
small, FT	14.8	30.6	21.4	23.8	19.0	22.9
base, FT+L	20.3	39.2	26.1	27.4	21.1	22.5
small, FT+L	**14.6**	**29.8**	**20.6**	**21.3**	**17.1**	**18.7**

Table 4: Test error (%) at 16-bit, greedy decoding. E: embedded-word error. FT = ordinary fine-tuning; FT+L = fine-tuning with $\mathcal{L}_{\text{role}} + \lambda\mathcal{L}_{\text{sem}}$. Speaker-cluster 95% CIs for small, FT+L: NSC Same Room MER [18.6, 23.4], Different Room [15.7, 18.6].

	ASCEND		NSC Same Room		NSC Different Room	
System	MER	E	MER	E	MER	E
(a) Our losses vs. ordinary fine-tuning (FT) of the same size						
base, role-weighted only	−0.7	−14.6	−0.6	**−2.7**	−0.6	**−0.9**
base, semantic only	−1.2	−14.6	**−1.3**	−1.7	**−1.1**	−0.4
base, FT+L (both)	−1.0	−16.1	**−2.3**	**−3.6**	**−1.3**	**−1.4**
small, role-weighted only	**−0.5**	−2.3	+0.1	−1.1	−0.9	−3.3
small, FT+L (both)	−0.1	−0.8	**−0.7**	**−2.4**	**−1.9**	**−4.2**
(b) On-device gated re-ranking vs. the same model's greedy output						
base, FT+L	**−0.7**	**−5.4**	**−0.4**	−0.4	**−0.3**	−0.7
small, FT+L	**−0.5**	**−3.9**	−0.1	+0.2	+0.1	−0.1

Table 5: Paired differences in points (negative = better). The two panels use different baselines. Bold: 95% CI of the difference excludes 0.

ter than FT on both NSC test sets, for base and small, in both MER and embedded error, and its critical-word error does not rise. These gains (0.7–2.3 MER points) are larger than the 0.2 points by which a retrain of the same system moved. On ASCEND the changes are mostly within noise: there are only two test speakers, and a few long runs of repeated text dominate the embedded error. On our phone recordings, FT+L and FT are within noise of each other (small, embedded: −3.6 [−10.0, 2.7]). A direct penalty on transliterations did not help (Appendix A.6).

Re-ranking helps only when gated. Re-ranking every clip lowers embedded error on Mandarin-main ASCEND (base FT+L, on-device: −4.3 points) but raises MER on English-main NSC (base FT+L, Different Room: +0.5; small FT+L, Same Room: +0.8). Gating fixes this (Table 5b). Only the 25–28% of clips where the language-ID head hears a switch are re-ranked. The result is never significantly worse than greedy decoding, and for base FT+L it is significantly better on all three test sets in MER. Without the repetition filter, re-ranking fails badly, because candidates that repeat themselves win the length-normalised score. The head labels 95–98% of speech frames correctly (Appendix A.2).

9 RQ3: Quantization

Answer: 8-bit is safe; 4-bit hurts embedded words more in tiny and base; our repairs work in NF4 but not in whisper.cpp's 4-bit format (H3 supported for tiny and base).

8-bit is safe. In every backend (LLM.int8, CTranslate2 int8, whisper.cpp Q8_0) and model size, 8-bit causes no detectable extra harm to embedded words: no 8-bit R has an interval above 1, and the matrix change is mostly within noise.

4-bit hurts embedded words more in tiny and base. For tiny and base, 4-bit raises embedded error 1.4 to 4.4 times as much as matrix error in the five settings where the interval of R excludes 1 (Table 6; e.g. tiny FT

Model	Format	Test set, main lang.	Δ_E	Δ_M	R
tiny, FT	NF4	ASCEND, Mandarin	+16.2	+4.7	3.47
base, FT	Q4_0	ASCEND, Mandarin	+8.2	+1.9	4.40
base, FT+L	Q4_0	ASCEND, Mandarin	+7.1	+2.3	3.07
base, FT+L	NF4	NSC Same, English	+3.0	+1.7	1.75
tiny, FT	Q4_0	NSC Same, English	+8.9	+6.4	1.40

Table 6: The five 4-bit settings whose 95% CI of R excludes 1, out of 20 reportable settings for tiny and base (FT and FT+L; NF4 and Q4_0). $R > 1$ in 14 of the 20 and the CI is below 1 in none. Δ: change in error from the same backend's 16-bit run, in points; "main lang." is the sentence's matrix language.

Repair	Model	Test set, main lang.	Loss	Rec [95% CI]
Mixed	base, FT	NSC Same, English	2.5	**0.86** [0.26, 2.99]
	base, FT	NSC Diff, English	3.5	−0.00 [−1.10, 0.34]
	tiny, FT	ASCEND, Mandarin	16.2	**0.65** [0.29, 0.91]
	tiny, FT	NSC Same, English	11.2	**0.77** [0.40, 1.05]
	tiny, FT	NSC Same, Mandarin	17.3	0.33 [−1.12, 0.78]
	tiny, FT	NSC Diff, English	6.4	**0.80** [0.28, 1.02]
	base, FT+L	NSC Same, English	3.0	**0.70** [0.38, 1.04]
	base, FT+L	NSC Same, Mandarin	2.6	0.10 [−1.55, 0.98]
	base, FT+L	NSC Diff, English	2.2	**0.48** [0.03, 0.90]
QLoRA	base, FT	NSC Same, English	2.5	−0.30 [−3.64, 2.08]
	base, FT	NSC Diff, English	3.5	0.65 [−0.75, 1.05]
	tiny, FT	ASCEND, Mandarin	16.2	**0.83** [0.57, 1.26]
	tiny, FT	NSC Same, English	11.2	**0.72** [0.46, 0.98]
	tiny, FT	NSC Same, Mandarin	17.3	**0.83** [0.39, 1.00]
	tiny, FT	NSC Diff, English	6.4	0.47 [−1.03, 0.92]
	base, FT+L	NSC Same, English	3.0	**0.77** [0.54, 1.10]
	base, FT+L	NSC Same, Mandarin	2.6	0.22 [−0.75, 0.75]
	base, FT+L	NSC Diff, English	2.2	0.10 [−0.51, 0.47]

Table 7: Recovery of the NF4 embedded-error loss, for every slice where the loss (points) has a CI excluding 0. Mixed precision keeps 1 (base FT), 2 (tiny FT) or 3 (base FT+L) blocks at 16-bit. Bold: CI of Rec excludes 0. After merging and re-quantizing to whisper.cpp Q4_0, no slice has a Rec CI above 0.

in NF4 on ASCEND: $R = 3.47$ [1.85, 6.13]). Across all 20 reportable tiny and base settings the direction is consistent: $R > 1$ in 14 and significantly below 1 in none. This holds for embedded English (ASCEND) and embedded Mandarin (NSC), and in both NF4 and Q4_0. Small is robust on the corpora: its errors change by about one point. In our per-block scan, the first encoder layer is always the most sensitive block.

The repairs work in NF4 but not in the on-device format. Keeping the two or three most sensitive blocks at 16-bit, or adding QLoRA adapters, recovers about half to four-fifths of the NF4 embedded-error loss where it helps (Table 7). For tiny FT and base FT+L, the recovery interval excludes 0 in 9 of the 14 affected slices. QLoRA does not help base FT, whose loss is small. When the adapters are merged into the weights and the model is re-quantized to whisper.cpp Q4_0, the recovery disappears: no slice has a Rec interval above 0 (for tiny FT on NSC Same Room, English-main: −0.12; base FT+L: 0.06). The adapters were trained against NF4's grid, not Q4_0's block-wise one, and whisper.cpp cannot load mixed-precision files at all. For whisper.cpp the answer is Q8_0, which changes embedded error by at most 1.2 points for any fine-tuned model.

	16-bit		Δ_E		
Model	MER	E	Q8_0	NF4	Q4_0
small, off-the-shelf	33.6	83.9			
tiny, FT	34.6	70.5	+0.9	+17.7	+5.1
base, FT	23.0	49.7	0.0	+5.5	**+8.9**
base, FT+L	24.4	50.7	−0.4	**+8.6**	**+7.2**
small, FT	14.3	31.4	−0.1	+4.1	−1.2
small, FT+L	13.4	27.8	+1.3	**+7.6**	+1.1

Table 8: Our phone recordings (120 utterances, 3 readers). Error (%) at 16-bit, and change in embedded error (points) to whisper.cpp Q8_0, NF4 and whisper.cpp Q4_0, each against its own backend's 16-bit run. Bold: 95% CI excludes 0 (utterance bootstrap; optimistic with 3 readers).

On our phone recordings, 4-bit also hurts embedded words. 8-bit again causes no detectable change: no 8-bit interval excludes 0 (Table 8), though with 227 embedded words the intervals reach up to about 4 points either side. At 4-bit, embedded error rises by 7–9 points for both base models (base FT, Q4_0: +8.9 [2.9, 15.4]) and for small FT+L in NF4 (+7.6 [2.6, 13.3]), one of the four small-model 4-bit settings, while matrix error mostly does not change measurably (small FT+L: −1.4 [−3.1, 0.3]). So R is mostly not reportable here. Tiny is dominated by repetition loops on single clips, so we draw no conclusion for it. These intervals are optimistic: there are three readers, and reader, phone and sentence are confounded.

Cost on a laptop. whisper.cpp Q8_0 is as fast as Q4_0, as accurate as F16 and about half its size (Table 9): small FT+L is a 252 MB file with RTF 0.57 and 20.3% MER. The language-ID head raises base's RTF from 0.21 to 0.30 (CTranslate2). Re-ranking every clip raises it to 1.76; with gating, which re-ranks about a quarter of clips, we estimate about

Model	Format	File	RSS	RTF	MER
tiny, FT	Q8_0	42	135	0.086	31.6
base, FT+L	Q8_0	78	196	0.176	25.8
base, FT+L	Q4_0	44	161	0.172	28.7
base, FT+L	F16	141	261	0.336	25.7
small, FT+L	Q8_0	252	427	0.573	20.3
small, FT+L	Q4_0	139	315	0.584	21.0
small, FT+L	CT2 int8	238	839	0.742	21.1
small, FT+L	F16	465	650	1.299	20.3

Table 9: Cost on a MacBook Pro (M2 Pro, 4 threads). File size and peak memory (RSS) in MB; RTF = processing time / audio time (below 1 is faster than real time); MER (%) on the NSC Same Room CPU subset. Formats are whisper.cpp unless marked CT2 (CTranslate2).

0.7 but did not measure it.

10 Example Outputs

Table 10 shows 16 of our phone recordings drawn at random. Off-the-shelf small keeps most embedded English in Mandarin-main sentences (insulin, module, release) but loses embedded Mandarin in English-main ones, writing it as English sound-alikes (身份证→“shuffling jump”, 贷款→“Taekwun”). Small FT+L fixes most of these but still fails on rare terms (布洛芬→不落分, 会议室→“hui yi shah”, prednisolone, polyclinic→polytechnic), and once repeats a phrase instead of the sentence. In NF4 most outputs are unchanged; it adds errors on 身份证 (written 身分证) and 期末考, and happens to avoid the repeated phrase.

11 Applications

We wrote our phone recordings around names, numbers, drug and course terms, and the clips in Table 10 point to settings where an on-device, code-switching recogniser is useful and where the embedded word carries the content:

- **Clinics and pharmacies.** Notes or patient instructions such as “医生说 prednisolone 要慢慢减量” or “the nurse said 布洛芬 only after meals”. Patient audio should stay on the device, and the drug name is exactly the word a wrong transcript must not lose.
- **Live captions for deaf and hard-of-hearing users** in lectures, tutorials and meetings, where course codes and technical terms (module, quiz, release) are embedded in Mandarin speech. Captions must keep up with speech: small FT+L in 8-bit runs at RTF 0.57 on a laptop CPU.
- **Banking, government and telco hotlines**, where callers mix in words like 身份证 or 贷款 over a phone line. Our phone recordings show that off-the-shelf Whisper writes these as English sound-alikes, while the fine-tuned models get them right.
- **Eldercare and family use**, e.g. voice reminders like “my mother keeps the 药 in the second drawer”. A home device should not stream household speech to the cloud, and it must still work without a fast network connection.
- **Personal dictation** of messages, notes and reminders (“the 期末考 starts on the first of December”), where a missed embedded word loses the one fact the note was written to keep.

Conversational use. The same recogniser can be the input to a spoken assistant: the user speaks, the on-device model transcribes each turn, a language model reads the transcript and writes a reply, and a speech synthesiser reads that reply aloud. In such a dialogue the embedded word is often the slot the assistant must act on (the drug, the date, the room), so the role-based error we measure matters more than overall MER. A dropped embedded word does not produce an obviously broken transcript. It produces a fluent sentence that the language model will answer confidently and wrongly (“bring your shuffling jump” instead of 身份证). Our language-ID head can also help here: it tells the assistant which language each part of the turn was in, so the reply can follow the user’s mix, and, because our best system still misses about one embedded word in five on NSC, the assistant can read back an embedded word before acting on it (“Prednisolone, is that right?”).

Turn-taking sets the speed budget. At RTF 0.57, a 5-second turn takes about 3 s to transcribe on our laptop, which is too slow for a natural reply; base at RTF 0.21 (CTranslate2) takes about 1 s, and gated re-ranking adds cost only on the quarter of turns with a switch. A conversational system would therefore pair a smaller or streaming recogniser with our loss

Reference (what was said)	Small, off-the-shelf, 16-bit	Small FT+L, 16-bit	Small FT+L, NF4
我们约了 polyclinic 下午三点半	我們約了POLITICNIC下午3點半 [1]	我们约了 polytechnic 下午 3 点半 [1]	我们圆了 polytechnic 下午 3 点半 [1]
护士说 vaccine 打完要等十五分钟	護士說, vaccine 打完要等 15 分鐘 [0]	抢了 15 分钟抢了 15 分钟 [1]	忽视说 vaccine 打完要等 15 分钟 [0]
这个 python script 跑了四十分钟	这个排队script跑了40分钟 [2]	这个 python script 跑了 40 分钟 [0]	这个 python script 跑了 40 分钟 [0]
你记得带 insulin 去，放在冰箱第二层	你記得帶 Insulin 區放在冰箱第二重 [0]	你记得带 insulin 去放在冰箱第二从 [0]	你记得带 insulin 去放在冰箱第二从 [0]
这个 module 的 quiz 占百分之二十	這個Module的Quiz,占20% [0]	这个 module 的 quiz 占百分之二十 [0]	这个 module 的 quiz 占百分之二十 [0]
这个 bug 要在 release 之前 fix	这个bug要在release之前fix。 [0]	这个 bug 要在 release 之前 fix [0]	这个 bug 要在 release 之前 fix [0]
医生说 prednisolone 要慢慢减量	醫生說,Pretinicillin要慢慢解釋 [1]	医生说 pratnicillin your 慢慢减凉 [1]	医生说 pratnicillin your 慢慢减量 [1]
那个 report 要写三千个字	那个report要写3000个字 [0]	那个 report 要写 3000 个字 [0]	那个 report 要写 3000 个字 [0]
the 药房 closes at nine tonight	The yellow phone closes at 9pm tonight. [2]	the 药房 closes at nine tonight [0]	the 药房 closes at nine tonight [0]
the nurse said 布洛芬 only after meals	The nurse said bull often only after meals. [3]	the nurse said 不落分 only after meals [3]	dinner set 不落分 only after meals [3]
he forgot the 门卡 so he could not enter level five	He forgot the Munkah so he could not answer level 5. [2]	he forgot the 门卡 so he could not answer level five [0]	he forgot the 门卡 so he could not answer level five [0]
please bring your 身份证 for the exam	Please remember to bring your shuffling jump for the exam. [3]	please remember to bring your 身份证 for the exam [0]	please remember to bring your 身分证 for the exam [1]
my mother keeps the 药 in the second drawer	My mother keeps the yao in the second drawer. [1]	my mother keeps the 药 in the second drawer [0]	my mother keeps the 药 in the second drawer [0]
I booked the 会议室 from two to four	I bought the Huyish from 2 to 4. [3]	i book the hui yi shah from two to four [3]	i book the hui yi shah from two to four [3]
the 贷款 has twelve monthly payments	The Taekwun has 12 monthly payments. [2]	the 贷款 has twelve monthly payments [0]	the 贷款 has twelve monthly payments [0]
the 期末考 starts on the first of December	The team who caused that on the 1st of December [3]	the 齐木考 starts on the first of december [2]	the 起目靠 start on the first of december [3]

Table 10: Sixteen of our phone recordings drawn at random (seed 425; top: Mandarin-main, bottom: English-main) and the raw outputs of three systems. Superscript: number of embedded words the system got wrong in that clip. Off-the-shelf Whisper often writes Traditional characters, which the normaliser maps to Simplified before scoring.

and gating methods, and could pass the previous turns to Whisper as a text prompt so that terms already mentioned in the dialogue are more likely to be recognised again. We have not tested these designs; they are directions this work makes possible, not results.

12 Conclusion

One overall error rate hides how speech recognisers treat the words that carry the meaning in code-switched speech. Splitting errors by role answers our three questions:

- **H1, supported.** Embedded words fail more often than the same words in the matrix language. Fine-tuning closes this gap for embedded Mandarin but not for embedded English. Errors are rarely translations.
- **H2, partly supported.** Our two losses and gated language-ID re-ranking give small but reliable gains on NSC; on ASCEND and our recordings they are within noise.
- **H3, supported for tiny and base.** 4-bit hurts embedded words more than other words; our repairs work in NF4 but not in whisper.cpp's 4-bit format.

Recommendation: on device, use 8-bit. It caused no detectable harm in any backend, and in whisper.cpp it is as fast as 4-bit.

Future work: more speakers and unprompted switching; losses that act on every embedded word rather than on whole candidates; and repairs trained on the deployed format's own grid.

Individual Contributions

All four members contributed to the project.
Rui Cong: text normaliser, unit splitter and role labels; the role scorer, metrics, bootstrap intervals, R and Rec; data loaders (with Nibras); read 40 of our phone sentences; translation-label audit; checked §3.
Phoebe: training code: fine-tuning, the role-weighted and semantic losses; read 40 phone sentences; critical-word audit; checked §5 and §8.
Bridgette: forced alignment, language-ID head, N-best re-ranking and gating; alignment checks and speed harness; read 40 phone sentences; checked §6.
Nibras: team lead; NSC Part 4 access and preparation; quantization (bitsandbytes, CTranslate2 and whisper.cpp exports, sensitivity scan, QLoRA); ran the phone-recording tests; drafted this report.

Limitations

Prompted switching. NSC speakers were asked to switch languages, and they switch more often within an utterance and in longer spans than ASCEND speakers (§4), so their switching may be more deliberate than natural code-switching.

Small ASCEND test set. ASCEND test has two speakers, so its utterance-level intervals are optimistic and several ASCEND differences are within noise. NSC claims use speaker-cluster intervals.

Automatic labels. The translation, transliteration and critical-word labels, and the check for partner speech leaking into NSC recordings, come from two LLM annotators with adjudication or from signal rules, not from people. The transliteration threshold ($\theta = 0.1$) is loose: for long words, one shared consonant code can be enough. Agreement is reported in Appendix A.3 (translation vs. rest $\kappa = 0.84$; critical words $\kappa = 0.77$; leakage rule vs. annotator $\kappa = 0.29$). Two team members each audited a small sample, which we take as the final human check. On 26 random translation pairs, the human agreed with our labels on 58% [39, 77] ($\kappa = 0.15$ [0.00, 0.40]): neither side found a translation, but the human called 11 of the 13 pairs our labels call transliterations unrelated, so our transliteration rates are likely too high. On 50 critical-word candidates drawn from cases where the rule tagger and the LLM annotators disagreed, agreement with the adjudicated list was 66% [54, 78] ($\kappa = 0.32$ [0.12, 0.55]); the human was stricter, rejecting 15 of the 38 words the list marks critical. Both audits have one annotator and few items (95% bootstrap intervals over items), and the partner-leakage labels were not audited.

Small set of phone recordings. Our phone recordings (§4) are 120 read sentences from three readers, each reading different sentences once, so reader, phone and sentence are confounded, intervals are utterance bootstraps and optimistic, and most fragility ratios are not reportable. In 24 clips all fine-tuned models make the same error; besides shared misrecognitions (*opens*→*open*), these include homophones no recogniser can resolve (她/他) and normaliser mismatches (*level 3* vs. *three*) that slightly inflate every system's error. The earlier synthetic stress set (CosyVoice 2) is kept in the appendix for comparison only.

One training run per system. Each system was trained once. A retrain of the same base system moved test MER by at most 0.2 points, smaller than the gains we report, but we did not retrain small.

Scope of speed results. Speed was measured on one machine (Apple M2 Pro); the relative order of formats may differ on x86 or phone processors. Gated re-ranking speed is an estimate from its parts.

Ethics and Data Licences

We use NSC Part 4 under the Singapore Open Data Licence version 1.0, which asks for the following notice: *Contains information from the National Speech Corpus (Part 4) accessed on 14 October 2026 from the Infocomm Media Development Authority of Singapore (IMDA), which is made available under the terms of the Singapore Open Data Licence version 1.0 (https://data.gov.sg/open-data-licence).* We use the data for research only and do not suggest that IMDA endorses this work. ASCEND and FLEURS are used under their published licences, and

the synthetic stress-set voices were produced with CosyVoice 2 (Apache-2.0) from FLEURS prompts (CC-BY 4.0). The human stress recordings were made by three of the authors, who agreed to their use for this project; they are shared only with the course staff as part of the submission and are not released publicly. We collected no data from people outside the team. Speech recognition that runs on the device keeps audio off remote servers, which matters for the clinical and counselling uses that motivate this work; errors on names, numbers and drug terms in such settings can still cause harm, so outputs should be checked by a person where mistakes matter.

References

Alexis Conneau, Min Ma, Simran Khanuja, Yu Zhang, Vera Axelrod, Siddharth Dalmia, Jason Riesa, Clara E. Rivera, and Ankur Bapna. 2023. FLEURS: Few-shot learning evaluation of universal representations of speech. In *IEEE Spoken Language Technology Workshop (SLT)*.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Fangxiaoyu Feng, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang. 2022. Language-agnostic BERT sentence embedding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Thomas Palmeira Ferraz. 2024. Efficient compression of multitask multilingual speech models. *arXiv preprint arXiv:2405.00966*.

Georgi Gerganov et al. whisper.cpp. `https://github.com/ggml-org/whisper.cpp`.

Ulf Hermjakob, Jonathan May, and Kevin Knight. 2018. Out-of-the-box universal romanization tool uroman. In *Proceedings of ACL 2018, System Demonstrations*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Jia Xin Koh, Aqilah Mislan, Kevin Khoo, Brian Ang, Wilson Ang, Charmaine Ng, and Yingying Tan. 2019. Building the Singapore English national speech corpus. In *Proceedings of Interspeech*.

Holy Lovenia, Samuel Cahyawijaya, Genta Indra Winata, Peng Xu, Yan Xu, Zihan Liu, Rita Frieske, Tiezheng Yu, Wenliang Dai, Elham J. Barezi, Qifeng Chen, Xiaojuan Ma, Bertram Emil Shi, and Pascale Fung. 2022. ASCEND: A spontaneous Chinese-English dataset for code-switching in multi-turn conversation. In *Proceedings of the Language Resources and Evaluation Conference (LREC)*.

MDBG. CC-CEDICT Chinese-English dictionary. `https://cc-cedict.org`.

OpenNMT. CTranslate2. `https://github.com/OpenNMT/CTranslate2`.

Puyuan Peng, Brian Yan, Shinji Watanabe, and David Harwath. 2023. Prompting the hidden talent of web-scale speech models for zero-shot task generalization. In *Proceedings of Interspeech*.

Rohit Prabhavalkar, Tara N. Sainath, Yonghui Wu, Patrick Nguyen, Zhifeng Chen, Chung-Cheng Chiu, and Anjuli Kannan. 2018. Minimum word error rate training for attention-based sequence-to-sequence models. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.

Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Sayani Kundu, Ali Elkahky, Zhaoheng Ni, Apoorv Vyas, Maryam Fazel-Zarandi, Alexei Baevski, Yossi Adi, Xiaohui Zhang, Wei-Ning Hsu, Alexis Conneau, and Michael Auli. 2024. Scaling speech technology to 1,000+ languages. *Journal of Machine Learning Research*, 25(97):1–52.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. ArXiv:2212.04356.

Arthur Söhler, Julian Irigoyen, and Andreas Søeborg Kirkedal. 2026. Quantizing Whisper: How design choices affect ASR performance. In *Proceedings of the SPEAKABLE Workshop at LREC 2026*.

Enes Yavuz Ugan, Ngoc-Quan Pham, Leonard Bärmann, and Alexander Waibel. 2025a. PIER: A novel metric for evaluating what matters in code-switching. *arXiv preprint arXiv:2501.09512*.

Enes Yavuz Ugan, Ngoc-Quan Pham, and Alexander Waibel. 2025b. Adapting language balance in code-switching speech. *arXiv preprint arXiv:2510.18724*.

Yuhang Yang, Yizhou Peng, Hao Huang, Eng Siong Chng, and Xionghu Zhong. 2024. Adapting OpenAI's Whisper for speech recognition on code-switch Mandarin-English SEAME and ASRU2019 datasets. In *Proceedings of the Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*.

Jiaming Zhou, Yujie Guo, Shiwan Zhao, Haoqin Sun, Hui Wang, Jie Ying He, Aobo Kong, Shiyao Wang, Xi Yang, Yequan Wang, Yonghua Lin, and Yong Qin. 2025. CS-Dialogue: A 104-hour dataset of spontaneous Mandarin-English code-switching dialogues for speech recognition. *arXiv preprint arXiv:2502.18913*.

A Additional Results

A.1 Hyperparameter selection

Grids were run on base and scored on pooled ASCEND + NSC validation (Table 11). Role weights: lowest embedded error among runs whose MER is at most 1 point above plain fine-tuning. Semantic loss: lowest semantic MER among runs whose critical-word error does not rise.

Run	MER	E	CWER	Chosen
plain fine-tuning	31.42	36.13	27.91	
α=0.5, β=0	31.57	35.04	28.35	
α=0.5, β=0.5	30.44	33.50	27.91	✓
α=1, β=0	34.34	44.82	27.47	
α=1, β=0.5	31.28	33.90	29.45	
α=2, β=0	32.49	33.69	28.57	
α=2, β=0.5	32.81	37.02	29.01	
λ=0.1, ρ_0=0.2	29.41	33.02	25.49	
λ=0.1, ρ_0=0.5	29.39	33.22	26.81	✓
λ=0.3, ρ_0=0.2	29.62	33.33	27.03	
λ=0.3, ρ_0=0.5	29.44	33.06	26.59	

Table 11: Validation grids on base (%). Semantic-loss runs use $\alpha = \beta = 0.5$.

A.2 Language-ID head

Head	Accuracy (%)	Boundary (ms)	Found (%)
shared (base FT)	96.0 / 95.4	60 / 120	75 / 85
own, base FT+L	95.9 / 95.3	60 / 100	77 / 84
own, small FT+L	97.7 / 96.4	20 / 80	78 / 85

Table 12: Language-ID head on validation (ASCEND / NSC). Accuracy on speech frames; median distance from a true switch to the nearest predicted one; share of switches detected.

The model's own head gives the same reranking accuracy as the shared head; its benefit is speed, because it reuses the model's encoder pass.

A.3 Label agreement and the translation threshold

Two LLM annotators labelled each item independently, the second seeing only the written rubric. On 1,768 ASCEND pairs, agreement was 0.798 ($\kappa = 0.62$) over three classes and 0.981 ($\kappa = 0.84$) for translation vs. the rest. On 335 candidate critical-word types, agreement was 0.899 ($\kappa = 0.77$); the rule tagger agreed less with either annotator ($\kappa = 0.55$ and 0.47), mostly because it has no named-entity component. Human audits of small samples agree less with these labels ($\kappa = 0.15$ for translation labels, $n = 26$; $\kappa = 0.32$ for critical words, $n = 50$; see Limitations); results table 20c lists every comparison with its interval. With τ between 0.6 and 0.7 the number of flagged translation occurrences changes by about ±20%, and translations remain a small share of embedded errors for every $\tau \leq 0.7$.

A.4 Partner speech and alignment

In NSC Same Room recordings the partner's voice is sometimes audible. A signal rule (overlap with the partner's transcript, speaker-embedding similarity, and output words that appear in the partner's transcript) flags 1.75% of Same Room and 0.15% of Different Room test utterances. Removing them lowers Same Room MER by about one point for every fine-tuned model (small FT+L: 20.65 → 19.71) and changes no conclusion. Forced-alignment boundaries are within a median of 10 ms of known switches in FLEURS splices (87% within 50 ms) and a median of 73 ms from Whisper's cross-attention times at real switches.

A.5 Training on Different Room audio

NSC training data (40 h)	Same test	Diff (held-out)
Same Room (= base FT)	28.4	21.6
Different Room	32.3	23.6
20 h Same + 20 h Diff	28.3	22.2

Table 13: MER (%) of base when NSC training audio comes from the Different Room condition (Different Room split by session; ASCEND training data unchanged).

Training on Different Room audio does not help on Different Room test speech: training

only on it is 2.0 points worse [0.4, 3.8] (Table 13). The gap between the two rooms in Table 4 is therefore not a mismatch between training and test conditions.

A.6 Transliteration penalty

To target transliterations directly, we added an extra cost to each candidate of the semantic loss:

$$c_{\text{tl}}(y) = c(y) + \gamma\, N_{\text{tl}}(y), \tag{6}$$

where $c(y)$ is the semantic cost (§5), $N_{\text{tl}}(y)$ is the number of embedded reference words that y writes as a sound-alike in the other script (§4), and γ is the extra weight; $\gamma = 0$ gives the semantic loss back. We tested it on base in two rounds: $\gamma \in \{0.5, 1\}$ at $\lambda = 0.1$, then $\lambda \in \{0.4, 1\} \times \gamma \in \{0, 3\}$. Each round included a $\gamma = 0$ retrain to measure run-to-run noise (the "control" rows of Table 14). The penalty did not lower transliteration, MER or embedded error on any test set. Its gradient was about 1/130 of the role-weighted loss's, because it acts only when a sampled candidate contains a transliteration; raised to match the main loss ($\lambda = 1$, $\gamma = 3$), it still did not beat FT+L. At $\lambda = 0.1$, no γ passed the validation rule (MER at most 0.5 points and critical-word error no higher than FT+L), so small was not trained. In the second round the rule allowed critical-word error up to 1 point higher, the size of the first round's retrain noise; only $\lambda = 1, \gamma = 3$ qualified, although it had the highest validation transliteration rate.

	MER			Transliteration rate		
Run	A	S	D	A	S	D
λ=0.1, γ=0 (control)	−0.14	−0.09	−0.08	+0.91	−0.07	+0.21
λ=0.1, γ=0.5	−0.27	+0.17	**+0.31**	+0.13	+0.05	−0.39
λ=0.1, γ=1	−0.37	−0.04	−0.02	−0.65	−0.25	−0.05
λ=1, γ=0 (control)	−0.12	−0.06	+0.16	+0.39	−0.31	−0.05
λ=1, γ=3 (chosen)	+0.12	+0.26	+0.19	+0.26	−0.04	−0.27

Table 14: Transliteration penalty on base: paired test differences vs. base FT+L (points; negative = better). A = ASCEND, S = NSC Same Room, D = NSC Different Room. Bold: CI excludes 0. Embedded error changes are not significant for any penalised run.

A.7 Synthetic stress set

We synthesised 120 code-switched sentences rich in critical words with CosyVoice 2 in four voices, clean and through a simulated phone channel (300–3400 Hz, 8 kHz, room reverberation, 20 dB noise). The phone channel roughly doubles errors and embedded words suffer most (small FT+L: MER 22.3% clean, 36.8% phone; embedded error 54.6% and 74.5%). Embedded error is high even on clean speech: the same sentences read by people give 15–27 points lower embedded error for every fine-tuned model (§9), which confirms that the synthetic voices often mispronounce embedded words. This set therefore supports only relative comparisons.

A.8 Logistic regression for RQ1

To separate role from language, we fit

$$\begin{aligned}\operatorname{logit} P(\text{err}) = {} & b_0 + b_{\text{emb}}\,\text{emb} + b_{\text{en}}\,\text{en} \\ & + b_{\text{emb}\times\text{en}}\,\text{emb}\cdot\text{en} \\ & + b_f \log f + b_d\, d + b_c\,\text{nsc},\end{aligned} \tag{7}$$

where emb, en and nsc are 1 if the unit is embedded, is English, or comes from NSC (else 0); f is the unit's frequency in the training data and d its spoken duration; and each b is a coefficient. It is fit on 46,916 units from 122 speakers, with CIs from a speaker bootstrap (Table 16). b_{emb} is the effect of being embedded for a Chinese character; for an English word it is $b_{\text{emb}} + b_{\text{emb}\times\text{en}}$.

A.9 How to run

Table 15 lists the commands, run from the root of `https://github.com/nibess1/cs425_codeswitch`. *Pre-processing* turns each corpus into 16 kHz FLAC clips and a JSONL manifest (raw and normalised reference, speaker, main language, duration). *Training* reads a YAML config whose fields can be overridden on the command line (Table 2). *Testing* decodes the test sets with a 16-bit, simulated 8/4-bit (bitsandbytes) or exported CPU model (CTranslate2, whisper.cpp) and writes one JSONL line per utterance. *Scoring* (`score.py`) computes every metric of §3 with bootstrap intervals.

```
# 0. Environment: Python 3.11, one CUDA GPU (CPU formats need ctranslate2 and whisper.cpp)
micromamba create -n cs425 -c conda-forge python=3.11 pip ffmpeg && micromamba activate cs425
pip install -r requirements.txt && pip install -e . && pytest        # normaliser + scorer tests
# 1. Pre-processing: 16 kHz mono FLAC clips + JSONL manifests in data/<corpus>/manifests/
huggingface-cli download CAiRE/ASCEND --repo-type dataset --local-dir data/ascend/hf
python scripts/prepare_ascend.py                       # official splits
python scripts/fetch_nsc_dropbox.py fetch data/nsc_part4 -j 2 --only cs-chn && python scripts/prepare_nsc.py
python scripts/prepare_fleurs.py                       # spliced clips for the language-ID head
python scripts/prepare_stress.py                       # our recordings: data/stress/audio/<reader>/<id>.wav
python scripts/align.py ascend_train nsc_train ascend_val nsc_val    # MMS word times
# 2. Training (checkpoints in ckpt/<run>/best)
python scripts/train.py configs/ft_base.yaml           # FT; also ft_tiny.yaml, ft_small.yaml
python scripts/train.py configs/ft_base.yaml --run full_base --alpha 0.5 --beta 0.5 --lambda_sem 0.1 --rho0 0.5
python scripts/train_lid.py --encoder ckpt/ft_base/best --run lid_ft_base
python scripts/train.py configs/ft_base.yaml --run qat_full_base --model ckpt/full_base/best \
       --lora true --lr 1e-4 --epochs 1 --warmup 100
# 3. Testing (predictions in preds/<model>__<precision>__<method>.jsonl)
python scripts/decode.py --model ckpt/full_base/best --model_tag full_base --precision nf4_bnb --method full
python scripts/deploy_eval.py --model ckpt/full_base/best --tag full_base --method full \
       --formats ct2_f32 ct2_int8 ggml_f16 ggml_q8_0 ggml_q4_0
python scripts/sensitivity.py --model ckpt/full_base/best --tag full_base      # mixed-precision blocks
python scripts/rerank.py --model ckpt/full_base/best --tag full_base --lid ckpt/lid_ft_base/head.pt --loop_filter
# 4. Scoring and tables (results/tables/*.csv, results/figures/)
python scripts/score.py full_base__fp16__full full_base__nf4_bnb__full     # all metrics, 95% CIs
python scripts/score.py --fragility full_base__fp16__full full_base__nf4_bnb__full --set nsc_test --main en
python scripts/make_results.py && python scripts/export_tables.py && python scripts/make_figures.py
GPU=0 bash scripts/pipeline.sh stress_human && python scripts/make_human_tables.py   # phone recordings
```

Table 15: Commands to pre-process the data, train, test and score (run from the repository root; GPU chosen with `CUDA_VISIBLE_DEVICES`). `scripts/pipeline.sh <stage>` chains these steps with the settings we used; `DRY=1` prints the commands.

System	b_{emb}	$b_{\text{emb}\times\text{en}}$
small, off-the-shelf	**0.94**	0.09
base, off-the-shelf	**1.18**	0.23
base, FT	0.01	**0.65**
small, FT	−0.07	**0.79**

Table 16: Logistic-regression coefficients (log-odds). Bold: 95% CI excludes 0.